

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

КУРСОВАЯ РАБОТА
по дисциплине «Программирование»
Тема: Обработка изображений

Студентка гр. 2304

Иванова М.А.

Преподаватель

Государкин Я.С.

Санкт-Петербург

2023

ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ

Студентка Иванова М.А.

Группа 2304

Тема работы: обработка изображений

Исходные данные:

Программе подается файл формата BMP без сжатия, глубиной цвета 24 бит, и команды для его обработки, предусмотренные CLI.

Содержание пояснительной записки:

Аннотация, Содержание, Введение, Создание программы, Тестирование, Инструкция по запуску, Заключение, Список использованных источников, Приложение А.

Предполагаемый объем пояснительной записки:

Не менее 20 страниц.

Дата выдачи задания: 22.03.2023

Дата сдачи реферата: 24.05.2023

Дата защиты реферата: 26.05.2023

Студентка

Иванова М.А.

Преподаватель

Государкин Я.С.

АННОТАЦИЯ

Курсовая работа представляет собой программу, обрабатывающую изображение формата BMP. Программа имеет CLI для пользовательского взаимодействия.

Программа реализует весь следующий функционал по обработке bmp-файла:

1. Отражение заданной области. Функционал определяется выбором оси, относительно которой отражать (горизонтальная или вертикальная), координатами левого верхнего угла области, координатами правого нижнего угла области.
2. Копирование заданной области. Функционал определяется координатами левого верхнего угла области-источника, координатами правого нижнего угла области-источника, координатами левого верхнего угла области-назначения.
3. Заменяет все пиксели одного заданного цвета на другой цвет. Функционал определяется цветом, который требуется заменить и цветом, на который требуется заменить.
4. Разделяет изображение на $N \times M$ частей. Функционал определяется количеством частей по оси X, количеством частей по оси Y, путем, куда сохранить кусочки.
5. Печать информации об изображении.
6. Печать справки о программе.

СОДЕРЖАНИЕ

Введение.....	5
1. Создание программы.....	6
1.1. Структура программы.....	6
1.2. Структуры.....	7
1.3. Работа с файлом изображения.....	7
1.4. Функции рисования.....	8
1.5. Обработка ошибок.....	9
1.6. Функция main и реализация CLI.....	9
2. Тестирование.....	11
2.1. Тестирование функционала программы.....	11
2.2. Проверка обработки ошибок.....	13
3. Инструкция по запуску.....	14
Заключение.....	15
Список использованных источников.....	16
Приложение А. Исходный код программы.....	17

ВВЕДЕНИЕ

Цель работы: создать программу для обработки изображений формата BMP в соответствии с заданием. Реализовать CLI для пользовательского взаимодействия.

1. СОЗДАНИЕ ПРОГРАММЫ

1.1. Структура проекта

Код проекта был разбит на следующие файлы:

- Main.c – точка входа в приложение. в нем содержатся главная функция и все вспомогательные функции для реализации CLI.
- Errors.h – заголовочный файл, содержащий информацию о видах ошибок и определений функций по их выводу.
- Structs.h – заголовочный файл, описывающий структуры, представляющие BMP файл и вспомогательные структуры.
- Iobmp.h – заголовочный файл, содержащий функции, связанные с открытием, генерацией и сохранением изображений и очисткой памяти.
- Iobmp.c – файл кода, содержащий описание функций, определенных в files.h.
- Functions.h – заголовочный файл, содержащий определение функций по обработке изображений.
- Functions.c – файл кода с описанием функций из funcs.h.

Также в проекте присутствует Makefile, упрощающий сборку этого проекта.

1.2. Структуры

В проекте реализованы следующие структуры:

- Header – хранит информацию о поле Header изображения;
- InfoHeader – хранит информацию о поле InfoHeader изображения;
- RGB – хранит информацию о пикселе изображения;

- BMP – хранит информацию обо всем изображении;
- ImageData – хранит матрицу пикселей, ее ширину и высоту;
- Coordinates – хранит пару координат;
- CoordinatesArrays – хранит набор из четырех массивов пар координат.

1.3. Работа с файлом изображения

В файле `iobmp.c` реализованы функции, связанные с открытием, генерацией и сохранением изображений и очисткой памяти.

Функция `freeImageData(RGB **image_data, size_t height)` предназначена для очистки памяти, выделенной под матрицу пикселей.

Функция `freeImage(BMP *image)` предназначена для очистки памяти, выделенной под хранение изображения.

Функция `freeCoordinatesArrays(CoordinatesArrays *coordinates_arrays)` предназначена для очистки памяти, выделенной под хранение набор из четырех массивов пар координат.

Функция `countBitsToSkip(BMP *image)` предназначена для расчета количества байтов выравнивания в изображении. Функция возвращает количество байтов выравнивания в строке пикселей.

Функция `countBitsToAdd(size_t width, size_t height)` предназначена для расчета количества байтов выравнивания, которые необходимы для генерации изображения по заданной ширине и высоте. Функция возвращает количество байтов выравнивания в строке пикселей.

Функция `checkInfoHeader(BMP *image)` предназначена для проверки поля `InfoHeader`: изображение должно быть третьей версией BMP, быть несжатым и иметь 24 бита на пиксель. В случае несоответствия функция выводит сообщение

на экран и возвращает ошибку.

Функция `openImage(const char *path)` предназначена для открытия BMP файла по заданному пути. Функция открывает файл, заполняет все поля структуры BMP значениями, взятыми из изображения. Функция возвращает указатель на структуру BMP. В случае возникновения ошибки, функция заполняет глобальную переменную кодом ошибки и возвращает NULL.

Функция `saveImage(BMP *image, const char *path)` предназначена для сохранения BMP файла по заданному пути. Функция сохраняет изображения, записывая все его поля в файл. Функция возвращает код возникшей ошибки или 0, если сохранение прошло успешно.

Функция `generateBMP(RGB **buffer, size_t width, size_t height)` предназначена для генерации изображений по заданной матрице пикселей, ее ширине и высоте. Функция возвращает указатель на структуру BMP. В случае возникновения ошибки, функция заполняет глобальную переменную кодом ошибки и возвращает NULL.

1.4. Функции обработки изображения

В файле `functions.c` реализованы функции, связанные с обработкой изображения.

Функция `help()` предназначена для вывода справки о работе с программой через командную строку.

Функция `info(BMP *image, const char *path)` предназначена для вывода информации об изображении по изображению и пути к нему: название файла, его размер, ширина, высота, количество бит на пиксель, сжатие и версия BMP. Для получения имени файла вызывается функция `getName(const char *path)`, которая по пути к файлу определяет его имя. Для получения размера файла вызывается функция `getSize(BMP *image, int *flag_Kb, int *flag_Mb)`, которая определяет размер файла в килобайтах или мегабайтах.

Функция `changeImageColor(BMP *image, RGB color, RGB new_color)` предназначена для изменения одного цвета изображения на другой. Для проверки соответствия цветов пикселей вызывается функция `RGBcmp(RGB elem1, RGB elem2)`, которая возвращает 1, если пиксели одинаковы и 0 в противном случае. Если цвет, который требовалось изменить, не найден в изображении, функция выводит сообщение на экран.

Функция `copypasteImageArea(BMP *image, Coordinates *copy_left, Coordinates *copy_right, Coordinates *paste_left)` предназначена для копирования и вставки области, заданной координатами. Для проверки корректности координат вызываются функции `checkCopyCoordinates(BMP *image, Coordinates *left, Coordinates *right)` и `checkPasteCoordinates(BMP *image, Coordinates *paste_left, Coordinates copy_left, Coordinates copy_right, size_t *start_y)`. Сначала производится копирование заданной области с помощью функции `copyImageArea(BMP *image, Coordinates left, Coordinates right)`, а потом ее вставка с помощью функции `pasteImageArea(BMP *image, ImageData *buffer, Coordinates left, size_t right_y)`. Если на каком-то из этапов произошла ошибка, функция вернет ее код.

Функция `splitImage(BMP *image, const char *path, const char *output_dir, size_t N, size_t M)` предназначена для разбиения изображения на части. Для проверки корректности вводимых данных вызывается функция `checkNM(BMP *image, size_t N, size_t M)`. Вызывается вспомогательная функция расчета координат `countCoordinates(BMP *image, size_t N, size_t M)`. Если на каком-то из этапов произошла ошибка, функция вернет ее код.

Функция `reflectImageArea(BMP *image, const char *axis, Coordinates *left, Coordinates *right)` предназначена для отображения изображения относительно вертикальной/горизонтальной оси. В зависимости от выбранной оси вызывается функция `reflectImageHorizontally(ImageData *image_data)` или `reflectImageVertically(ImageData *image_data)`. Если на каком-то из этапов произошла ошибка, функция вернет ее код.

1.5. Обработка ошибок

В файле `error.h` реализован `enum Error` – коды ошибок, используемых в работе, а также массив строк `err_desc`, содержащий расшифровки кодов. Функция `print_error()` получает на вход код ошибки и выводит в поток ошибок соответствующее сообщение. Глобальная переменная `Error some_kind_of_image_error` хранит код ошибки, если функция, в которой произошла ошибка, возвращает указатель.

В случае ошибки программа очищает память, выводит сообщение в `stderr` и завершает свою работу с кодом вызванной ошибки.

Для очищения памяти и аварийного выхода созданы функции `safeInputErrorExit(const char *input_path, const char *output_path, char* axis)` и `safeFuncErrorExit(Error error, const char *input_path, const char *output_path, char* axis)`, помогающие избежать дублирования кода.

1.6. Функция `main` и реализация CLI

В файле `main.c` помимо точки входа присутствует функция `help()`, выводящая подсказку пользователю.

В функции `main` при помощи цикла и `getopt_long()` считываются флаги командной строки. Можно выбрать только одну функцию обработки за раз. Обрабатываются следующие флаги:

- `-H, --help` – вывод справки по работе с программой;
- `-I, --info` – вывод информации об изображении по работе с программой;
- `-R, --reflect` – отражение изображения;
- `-P, --copypaste` – копирование и вставка заданной области;
- `-C, --color` – замена цвет изображения;
- `-S, --split` – деление изображения на части;
- `-i, --input` – путь входного файла;
- `-o, --output` – путь выходного файла;

- -a, --axis – ось для отражения;
- -l, --upper_left – координаты левого верхнего угла области-источника;
- -r, --lower_right – координаты правого нижнего угла области-источника;
- -c, --old_color – цвет, который нужно заменить;
- -n, --new_color – цвет, на который нужно заменить;
- -x, --x_parts – количество частей для деления по оси x;
- -y, --y_parts – количество частей для деления по оси y;

Если пользователь не хочет менять исходное изображение, то он может сохранить результат работы функции по введенному им пути с помощью опции -o, --output.

Если в командную строку было передана функция и достаточное количество аргументов верного формате, происходит открытие изображения и работа с ним. После завершения работы с изображением программа очищает выделенную память и завершает свою работу.

Если пользователь не ввел функцию или ввел недопустимые значения, аргументы без функции, недостаточное количество аргументов, неправильный формат аргументов при вызове функции или вызвал несколько функций – программа вернет ошибку. В случае запуска программы без аргументов выводится справка.

2. ТЕСТИРОВАНИЕ

2.1. Тестирование функционала программы

Корректность работы программы по замене цвета:

```
--color --old_color 254:254:254 --new_color 0:0:0 --input /mnt/c/Rainbow.bmp
```



Рис. 1.1. Исходное изображение



Рис. 1.2. Замена цвета

Корректность работы программы по копированию и вставке заданной области:

```
-P -l 500:200 -r 650:0 -p 400:550 -i /mnt/c/Poppies.bmp
```



Рис. 2.1. Исходное изображение



Рис. 2.2. Копирование и вставка

Корректность работы программы по отражению заданной области:

```
--reflect --copy_left 400:350 --copy_right 550:200 --axis h --input /mnt/c/Poppies.bmp
```



Рис. 3.1. Исходное изображение



Рис. 3.2. Отражение области

Корректность работы программы по делению изображения на части:

```
-S -x 2 -y 2 -i /mnt/c/Poppies.bmp -o /mnt/c
```



Рис. 3.1. Исходное изображение



Рис. 3.2. Деление изображения

2.2. Проверка обработки ошибок

Корректность вывода информации об ошибке представлена на рисунке 8.

```
problem with --end  
Value error - incorrect arguments
```

Рис. 4. Вывод ошибки

3. ИНСТРУКЦИЯ ПО ЗАПУСКУ

Сборка программы происходит следующим образом:

- Вызов команды `make` – сборка программы
- Вызов команды `./cw_bmp` – запуск программы
- Ознакомиться со справкой об использовании
- Использовать программу

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была разработана программа, считывающая и обрабатывающая аргументы командной строки и BMP файлы. Программа была разбита на несколько файлов, а также был написан Makefile.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Алгоритм Брезенхэйма для отрезка и окружности // Википедия – свободная энциклопедия. URL:
https://ru.wikipedia.org/wiki/Алгоритм_Брезенхэма (дата обращения 01.05.2023).
2. BMP file format // Википедия – свободная энциклопедия. URL:
https://en.wikipedia.org/wiki/BMP_file_format (дата обращения: 15.12.2022).
3. Разбор командной строки // Habr.com. URL:
<https://habr.com/ru/articles/55665/> (дата обращения: 15.05.2023).

ПРИЛОЖЕНИЕ А
ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл errors.h:

```

#ifndef ERRORS_H
#define ERRORS_H

typedef enum {
    NO_ERROR = 0,
    ERROR_FILE_FORMAT = 1,
    ERROR_BMP_FORMAT = 2,
    ERROR_OPEN_FILE = 3,
    ERROR_READ_FILE = 4,
    ERROR_WRITE_FILE = 5,
    ERROR_MEMORY = 6,
    ERROR_INPUT = 7,

    ERROR_COUNT = 8,
} Error;

extern char *err_desc[ERROR_COUNT];

extern Error some_kind_of_image_error;

void safeInputErrorExit(const char *input_path, const char *output_path,
char* axis);

void safeFuncErrorExit(Error error, const char *input_path, const char
*output_path, char* axis);

#endif

```

Файл structs.h:

```

#ifndef STRUCTS_H
#define STRUCTS_H

#include <stdint.h>
#include <stdlib.h>
#include "errors.h"

#pragma pack (push, 1)

typedef struct {
    uint16_t Signature; // Тип файла (= 0x4D42 = BM)
    uint32_t FileSize; // Размер файла в байтах
    uint32_t reserved; // (= 0)
    uint32_t DataOffset; // Смещение к началу изображения в байтах
} Header; // 14 байт

typedef struct {
    uint32_t Size; // Размер структуры InfoHeader в байтах (= 40)
    uint32_t Width; // Ширина в пикселях
    uint32_t Height; // Высота в пикселях
    uint16_t Planes; // Количество плоскостей (= 1)
    uint16_t BitsPerPixel; // Количество битов на один пиксель (= 24)
    uint32_t Compression; // Метод сжатия (= 0 = BI_RGB)
    uint32_t ImageSize; // Размер изображения в байтах (может быть = 0)
    uint32_t XpixelsPerM; // Количество пикселей на метр по горизонтали
(может быть = 0 ???)
    uint32_t YpixelsPerM; // Количество пикселей на метр по вертикали (может
быть = 0 ???)
    uint32_t ColorsUsed; // Размер таблицы цветов (может быть = 0 для
максимально допустимой глубины цвета)

```

```

        uint32_t ImportantColors; // Количество цветов, необходимое для
        отображения файла без искажений (0 = все)
    } InfoHeader; // 40 байт

typedef struct {
    uint8_t Blue;
    uint8_t Green;
    uint8_t Red;
} RGB;

typedef struct {
    Header BitmapFileHeader;
    InfoHeader BitmapFileInfoHeader;
    RGB **data;
} BMP;

#pragma pack (pop)

typedef struct {
    uint32_t width;
    uint32_t height;
    RGB **data;
} ImageData;

typedef struct {
    size_t x;
    size_t y;
} Coordinates;

typedef struct {
    size_t *x_left;
    size_t *y_left;
    size_t *x_right;
    size_t *y_right;
} CoordinatesArrays;

#endif

```

Файл iobmp.h:

```

#ifndef IOBMP_H
#define IOBMP_H

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "errors.h"
#include "structs.h"

void freeImageData(RGB **image_data, size_t height);

void freeImage(BMP *image);

void freeCoordinatesArrays(CoordinatesArrays *coordinates_arrays);

size_t countBitsToSkip(BMP *image);

size_t countBitsToAdd(size_t width, size_t height);

```

```

Error checkInfoHeader(BMP *image);

BMP *openImage(const char *path);

Error saveImage(BMP *image, const char *path);

BMP *generateImage(RGB **buffer, size_t width, size_t height);

BMP *generatePlainColorImage(RGB color, size_t width, size_t height);

#endif

```

Файл iobmp.c:

```

#include "iobmp.h"

void freeImageData(RGB **image_data, size_t height) {
    for (size_t j = 0; j < height; j++) {
        free(image_data[j]);
    }
    free(image_data);
}

void freeImage(BMP *image) {
    freeImageData(image->data, image->BitmapFileInfoHeader.Height);
    free(image);
}

void freeCoordinatesArrays(CoordinatesArrays *coordinates_arrays) {
    free(coordinates_arrays->x_left);
    free(coordinates_arrays->y_left);
    free(coordinates_arrays->x_right);
    free(coordinates_arrays->y_right);
    free(coordinates_arrays);
}

size_t countBitsToSkip(BMP *image) {
    size_t width = image->BitmapFileInfoHeader.Width;
    size_t height = image->BitmapFileInfoHeader.Height;
    size_t file_size = image->BitmapFileHeader.FileSize;

    size_t real_data_size = file_size - sizeof(Header) - sizeof(InfoHeader);
    size_t theory_data_size = width * height * sizeof(RGB);
    size_t bits = (real_data_size - theory_data_size) / height;

    return bits;
}

size_t countBitsToAdd(size_t width, size_t height) {
    size_t theory_data_size = width * height * sizeof(RGB);
    size_t real_data_size;
    size_t num_1;
    size_t num_2;
    size_t bits;

    if (theory_data_size % (height * 4)) {

```

```

        num_1 = theory_data_size / 4;
        num_2 = (num_1 / height + 1) * height;
        real_data_size = num_2 * 4;
        bits = (real_data_size - theory_data_size) / height;

        return bits;
    }

    return 0;
}

Error checkInfoHeader(BMP *image) {
    if (image->BitmapFileInfoHeader.Size != 40) {
        printf("It's not the 3rd version of BMP!\n");
        return ERROR_BMP_FORMAT;
    }
    if (image->BitmapFileInfoHeader.BitsPerPixel != 24) {
        printf("It's not 24 bits per pixel!\n");
        return ERROR_BMP_FORMAT;
    }
    if (image->BitmapFileInfoHeader.Compression != 0) {
        printf("It's compressed!\n");
        return ERROR_BMP_FORMAT;
    }
    return NO_ERROR;
}

BMP *openImage(const char *path) {
    char *format;
    BMP *image;
    size_t extra_bits;
    size_t width;
    size_t height;
    size_t read;

    FILE *file = fopen(path, "rb");
    if (!file) {
        some_kind_of_image_error = ERROR_OPEN_FILE;
        return NULL;
    }

    format = strrchr(path, '.');
    if (!(format && strcmp(format, ".bmp") == 0)) {
        fclose(file);
        some_kind_of_image_error = ERROR_FILE_FORMAT;
        return NULL;
    }

    image = (BMP *) malloc(sizeof(BMP));
    if (!image) {
        fclose(file);
        some_kind_of_image_error = ERROR_MEMORY;
        return NULL;
    }

    rewind(file);

    read = fread(&image->BitmapFileHeader, sizeof(Header), 1, file);

```

```

    if (read != 1) {
        fclose(file);
        free(image);
        some_kind_of_image_error = ERROR_READ_FILE;
        return NULL;
    }
    if (image->BitmapFileHeader.Signature != 0x4D42) {
        fclose(file);
        free(image);
        some_kind_of_image_error = ERROR_FILE_FORMAT;
        return NULL;
    }

    read = fread(&image->BitmapFileInfoHeader, sizeof(InfoHeader), 1, file);
    if (read != 1) {
        fclose(file);
        free(image);
        some_kind_of_image_error = ERROR_READ_FILE;
        return NULL;
    }
    if (checkInfoHeader(image)) {
        fclose(file);
        free(image);
        some_kind_of_image_error = ERROR_BMP_FORMAT;
        return NULL;
    }

    read = 0;
    width = image->BitmapFileInfoHeader.Width;
    height = image->BitmapFileInfoHeader.Height;
    extra_bits = countBitsToSkip(image);

    image->data = (RGB **) malloc(sizeof(RGB *) * height);
    if (!image->data) {
        fclose(file);
        free(image);
        some_kind_of_image_error = ERROR_MEMORY;
        return NULL;
    }

    for (size_t j = 0; j < height; j++) {
        image->data[j] = (RGB *) malloc(sizeof(RGB) * width + extra_bits);
        if (!image->data[j]) {
            fclose(file);
            for (size_t k = 0; k < j; k++) {
                free(image->data[k]);
            }
            free(image->data);
            free(image);
            some_kind_of_image_error = ERROR_MEMORY;
            return NULL;
        }
        read += fread(image->data[j], sizeof(RGB) * width + extra_bits, 1,
file);
    }
    if (read != height) {
        fclose(file);
        freeImage(image);
        some_kind_of_image_error = ERROR_READ_FILE;
        return NULL;
    }

```

```

    }

    fclose(file);

    return image;
}

Error saveImage(BMP *image, const char *path) {
    char *format;
    size_t extra_bits;
    size_t width;
    size_t height;
    size_t write;

    FILE *file = fopen(path, "wb");
    if (!file) {
        return ERROR_OPEN_FILE;
    }

    format = strrchr(path, '.');
    if (!(format && strcmp(format, ".bmp") == 0)) {
        fclose(file);
        remove(path);
        return ERROR_FILE_FORMAT;
    }

    write = fwrite(&image->BitmapFileHeader, sizeof(Header), 1, file);
    if (write != 1) {
        fclose(file);
        remove(path);
        return ERROR_WRITE_FILE;
    }
    write = fwrite(&image->BitmapFileInfoHeader, sizeof(InfoHeader), 1,
file);
    if (write != 1) {
        fclose(file);
        remove(path);
        return ERROR_WRITE_FILE;
    }

    write = 0;
    width = image->BitmapFileInfoHeader.Width;
    height = image->BitmapFileInfoHeader.Height;
    extra_bits = countBitsToSkip(image);

    for (size_t j = 0; j < height; j++) {
        write += fwrite(image->data[j], sizeof(RGB) * width + extra_bits, 1,
file);
    }
    if (write != height) {
        fclose(file);
        remove(path);
        return ERROR_WRITE_FILE;
    }

    fclose(file);
    return NO_ERROR;
}

```

```

BMP *generateImage(RGB **buffer, size_t width, size_t height) {
    size_t extra_bits = countBitsToAdd(width, height);
    BMP *image = (BMP *) malloc(sizeof(BMP));
    if (!image) {
        some_kind_of_image_error = ERROR_MEMORY;
        return NULL;
    }

    image->BitmapFileHeader.Signature = 0x4D42;
    image->BitmapFileHeader.FileSize =
        sizeof(Header) + sizeof(InfoHeader) + sizeof(Header) * width *
height + extra_bits * height;
    image->BitmapFileHeader.reserved = 0;
    image->BitmapFileHeader.DataOffset = sizeof(Header) + sizeof(InfoHeader);

    image->BitmapFileInfoHeader.Size = 40;
    image->BitmapFileInfoHeader.Width = width;
    image->BitmapFileInfoHeader.Height = height;
    image->BitmapFileInfoHeader.Planes = 1;
    image->BitmapFileInfoHeader.BitsPerPixel = 24;
    image->BitmapFileInfoHeader.Compression = 0;
    image->BitmapFileInfoHeader.ImageSize = sizeof(Header) * width * height +
extra_bits * height; // 0
    image->BitmapFileInfoHeader.XpixelsPerM = 11811; // 300dpi // 4700 ???
    image->BitmapFileInfoHeader.YpixelsPerM = 11811; // 300dpi // 4700 ???
    image->BitmapFileInfoHeader.ColorsUsed = 0; // sizeof(char) ???
    image->BitmapFileInfoHeader.ImportantColors = 0; // sizeof(char) ???

    image->data = (RGB **) malloc(sizeof(Header) * height);
    if (!image->data) {
        free(image);
        some_kind_of_image_error = ERROR_MEMORY;
        return NULL;
    }

    for (size_t j = 0; j < height; j++) {
        image->data[j] = (Header *) calloc(1, sizeof(Header) * width + extra_bits);
        if (!image->data[j]) {
            for (size_t k = 0; k < j; k++) {
                free(image->data[k]);
            }
            free(image->data);
            free(image);
            some_kind_of_image_error = ERROR_MEMORY;
            return NULL;
        }
        memcpy(image->data[j], buffer[j], width * sizeof(Header));
    }

    return image;
}

BMP *generatePlainColorImage(Header color, size_t width, size_t height) {
    size_t extra_bits = countBitsToAdd(width, height);
    BMP *image = (BMP *) malloc(sizeof(BMP));
    if (!image) {
        some_kind_of_image_error = ERROR_MEMORY;
        return NULL;
    }

```

```

    }

    image->BitmapFileHeader.Signature = 0x4D42;
    image->BitmapFileHeader.FileSize =
        sizeof(Header) + sizeof(InfoHeader) + sizeof(Header) * width *
height + extra_bits * height;
    image->BitmapFileHeader.reserved = 0;
    image->BitmapFileHeader.DataOffset = sizeof(Header) + sizeof(InfoHeader);

    image->BitmapFileInfoHeader.Size = 40;
    image->BitmapFileInfoHeader.Width = width;
    image->BitmapFileInfoHeader.Height = height;
    image->BitmapFileInfoHeader.Planes = 1;
    image->BitmapFileInfoHeader.BitsPerPixel = 24;
    image->BitmapFileInfoHeader.Compression = 0;
    image->BitmapFileInfoHeader.ImageSize = sizeof(Header) * width * height +
extra_bits * height; // 0
    image->BitmapFileInfoHeader.XpixelsPerM = 11811; // 300dpi // 4700 ???
    image->BitmapFileInfoHeader.YpixelsPerM = 11811; // 300dpi // 4700 ???
    image->BitmapFileInfoHeader.ColorsUsed = 0; // sizeof(char) ???
    image->BitmapFileInfoHeader.ImportantColors = 0; // sizeof(char) ???

    image->data = (RGB **) malloc(sizeof(RGB *) * height);
    if (!image->data) {
        free(image);
        some_kind_of_image_error = ERROR_MEMORY;
        return NULL;
    }
    for (size_t j = 0; j < height; j++) {
        image->data[j] = (RGB *) calloc(1, sizeof(RGB) * width + extra_bits);
        if (!image->data[j]) {
            for (size_t k = 0; k < j; k++) {
                free(image->data[k]);
            }
            free(image->data);
            free(image);
            some_kind_of_image_error = ERROR_MEMORY;
            return NULL;
        }
        for (size_t i = 0; i < width; i++) {
            image->data[j][i].Red = color.Red;
            image->data[j][i].Green = color.Green;
            image->data[j][i].Blue = color.Blue;
        }
    }

    return image;
}

```

Файл functions.h:

```

#ifndef FUNCTIONS_H
#define FUNCTIONS_H

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "errors.h"
#include "structs.h"
#include "iobmp.h"

```

```

void help();

const char *getName(const char *path);

double getSize(BMP *image, int *flag_Kb, int *flag_Mb);

Error info(BMP *image, const char *path);

int RGBcmp(RGB elem1, RGB elem2);

void changeImageColor(BMP *image, RGB color, RGB new_color);

int checkCopyCoordinates(BMP *image, Coordinates *left, Coordinates *right);

int checkPasteCoordinates(BMP *image, Coordinates *paste_left, Coordinates
copy_left, Coordinates copy_right,
                        size_t *start_y);

ImageData *copyImageArea(BMP *image, Coordinates left, Coordinates right);

void pasteImageArea(BMP *image, ImageData *buffer, Coordinates left, size_t
right_y);

Error copypasteImageArea(BMP *image, Coordinates *copy_left, Coordinates
*copy_right, Coordinates *paste_left);

const char *makePath(const char *path, int part_image_num);

int checkNM(BMP *image, size_t N, size_t M);

CoordinatesArrays *countCoordinates(BMP *image, size_t N, size_t M);

Error splitImage(BMP *image, const char *path, size_t N, size_t M);

void reflectImageVertically(ImageData *image_data);

Error reflectImageHorizontally(ImageData *image_data);

Error reflectImageArea(BMP *image, const char *axis, Coordinates *left,
Coordinates *right);

#endif

```

Файл functions.c:

```
#include "functions.h"
```

```

void help() {
    printf("This program can change BMP images\n");
    printf("This program supports only 3rd version of BMP, 24 bits per pixel
and without compression!\n");
    printf("\n");
    printf("You can choose one option from the list at a time:\n");
    printf("1) Get information about image:\n");
    printf("\t Long command format: --info --input /mnt/input_path\n");
    printf("\t Short command format: -I -i /mnt/input_path\n");
    printf("\n");
    printf("2) Reflect image area:\n");
    printf("\t Long command format: --reflect --input /mnt/input_path --axis
horizontal --upper_left x:y --lower_right x:y\n");
    printf("\t Short command format: -R -i /mnt/input_path -a h -l x:y -r
x:y\n");
}

```

```

    printf("\t Note: --axis can be horizontal (h) or vertical (v) \n");
    printf("\n");
    printf("3) Copy and paste image area:\n");
    printf("\t Long command format: --copypaste --input /mnt/input_path --
upper_left x:y --lower_right x:y --paste x:y\n");
    printf("\t Short command format: -P -i /mnt/input_path -l x:y -r x:y -p
x:y\n");
    printf("\n");
    printf("4) Change image color:\n");
    printf("\t Long command format: --color --input /mnt/input_path --
old_color R:G:B --new_color R:G:B\n");
    printf("\t Short command format: -C -i /mnt/input_path -c R:G:B -n
R:G:B\n");
    printf("\n");
    printf("5) Split image:\n");
    printf("\t Long command format: --split --input /mnt/input_path --x_parts
x --y_parts y --output /mnt/output_path\n");
    printf("\t Short command format: -S -i /mnt/input_path -x x -y y -o
/mnt/output_path\n");
    printf("\n");
    printf("6) Get help and see this text again:\n");
    printf("\t Long command format: --help\n");
    printf("\t Short command format: -H\n");
    printf("\n");

    printf("If you don't want to change the input image by using options 2)-
4), you can add option --output /mnt/output_path or -o /mnt/output_path\n");
    printf("New image will be generated as a result of function work and
saved by this path\n");
}

const char *getName(const char *path) {
    char *slash = strrchr(path, '/');
    char *name = (char *) malloc((strlen(slash) - 4) * sizeof(char));
    if (!name) {
        some_kind_of_image_error = ERROR_MEMORY;
        return NULL;
    }
    memcpy(name, slash + 1, (strlen(slash) - 5) * sizeof(char));
    name[strlen(slash) - 5] = '\0';
    return name;
}

double getSize(BMP *image, int *flag_Kb, int *flag_Mb) {
    uint32_t file_size = image->BitmapFileHeader.FileSize;
    double file_size_2 = 0;
    if (file_size / 1024.0 > 1) {
        file_size_2 = file_size / 1024.0;
        *flag_Kb = 1;
        if (file_size_2 / 1024.0 > 1) {
            file_size_2 /= 1024.0;
            *flag_Mb = 1;
        }
    }
    return file_size_2;
}

```

```

Error info(BMP *image, const char *path) {
    int flag_Kb = 0;
    int flag_Mb = 0;
    double file_size_2;
    const char *name = getName(path);
    if (!name) {
        return some_kind_of_image_error;
    }

    printf("Information about image %s\n", path);
    printf("\tImage name: %s\n", name);
    printf("\tImage size: %u bytes", image->BitmapFileHeader.FileSize);
    file_size_2 = getSize(image, &flag_Kb, &flag_Mb);
    if (flag_Mb) {
        printf(" = %.2f Mb\n", file_size_2);
    }
    if (flag_Kb && !flag_Mb) {
        printf(" = %.2f Kb\n", file_size_2);
    }
    if (!flag_Kb && !flag_Mb) {
        printf("\n");
    }
    printf("\tImage width: %u pixels\n", image->BitmapFileInfoHeader.Width);
    printf("\tImage height: %u pixels\n", image->
>BitmapFileInfoHeader.Height);
    printf("\tImage bits per pixel: %u\n", image->
>BitmapFileInfoHeader.BitsPerPixel);
    printf("\tImage version of BMP: 3\n");
    printf("\tImage is not compressed\n");

    free((char *) name);
    return NO_ERROR;
}

int RGBcmp(RGB elem1, RGB elem2) {
    return (elem1.Red == elem2.Red) && (elem1.Green == elem2.Green) &&
(elem1.Blue == elem2.Blue);
}

void changeImageColor(BMP *image, RGB color, RGB new_color) {
    size_t width = image->BitmapFileInfoHeader.Width;
    size_t height = image->BitmapFileInfoHeader.Height;
    int flag = 0;

    for (size_t j = 0; j < height; j++) {
        for (size_t i = 0; i < width; i++) {
            if (RGBcmp(image->data[j][i], color)) {
                flag += flag == 0;
                image->data[j][i].Red = new_color.Red;
                image->data[j][i].Green = new_color.Green;
                image->data[j][i].Blue = new_color.Blue;
            }
        }
    }

    if (!flag) {
        printf("Color not found\n");
    }
}

```

```

}

int checkCopyCoordinates(BMP *image, Coordinates *left, Coordinates *right) {
    size_t width = image->BitmapFileInfoHeader.Width;
    size_t height = image->BitmapFileInfoHeader.Height;

    if (left->x >= width || right->y >= height) {
        return 0;
    }
    if (right->x >= width) {
        right->x = width - 1;
    }
    if (left->y >= height) {
        left->y = height - 1;
    }
    if (left->x > right->x || right->y > left->y) {
        return 0;
    }

    return 1;
}

```

```

int
checkPasteCoordinates(BMP *image, Coordinates *paste_left, Coordinates
copy_left, Coordinates copy_right,
                    size_t *start_y) {
    size_t width = image->BitmapFileInfoHeader.Width;
    size_t height = image->BitmapFileInfoHeader.Height;
    size_t delta_y = copy_left.y - copy_right.y + 1;
    int paste_right_y;

    if (paste_left->x >= width) {
        return 0;
    }
    if (paste_left->y >= height) {
        paste_left->y = height - 1;
    }
    paste_right_y = (int) (paste_left->y - delta_y + 1);
    if (paste_right_y < 0) {
        paste_right_y = 0;
    }
    *start_y = paste_right_y;

    return 1;
}

```

```

ImageData *copyImageArea(BMP *image, Coordinates left, Coordinates right) {
    ImageData *image_data;
    size_t width = image->BitmapFileInfoHeader.Width;
    size_t height = image->BitmapFileInfoHeader.Height;
    size_t delta_x = right.x - left.x + 1;
    size_t delta_y = left.y - right.y + 1;
    size_t u = 0;
    size_t v = 0;

    RGB **buffer = (RGB **) malloc(sizeof(RGB *) * delta_y);
    if (!buffer) {

```

```

        some_kind_of_image_error = ERROR_MEMORY;
        return NULL;
    }

    for (size_t j = right.y; (j < left.y + 1) && (j < height); j++) {
        buffer[v] = (RGB *) malloc(sizeof(RGB) * delta_x);
        if (!buffer[v]) {
            freeImageData(buffer, v);
            some_kind_of_image_error = ERROR_MEMORY;
            return NULL;
        }
        for (size_t i = left.x; (i < right.x + 1) && (i < width); i++) {
            buffer[v][u].Red = image->data[j][i].Red;
            buffer[v][u].Green = image->data[j][i].Green;
            buffer[v][u].Blue = image->data[j][i].Blue;
            u++;
        }
        u = 0;
        v++;
    }

    image_data = (ImageData *) malloc(sizeof(ImageData));
    if (!image_data) {
        freeImageData(buffer, v);
        some_kind_of_image_error = ERROR_MEMORY;
        return NULL;
    }
    image_data->width = delta_x;
    image_data->height = delta_y;
    image_data->data = buffer;

    return image_data;
}

void pasteImageArea(BMP *image, ImageData *buffer, Coordinates left, size_t
right_y) {
    size_t width = image->BitmapFileInfoHeader.Width;
    size_t height = image->BitmapFileInfoHeader.Height;
    size_t delta_x = buffer->width;
    size_t delta_y = buffer->height;
    size_t u = 0;
    size_t v = 0;

    for (size_t j = right_y; (j < left.y + 1) && (j < height) && (v < delta_y
+ 1); j++) {
        for (size_t i = left.x; (i < left.x + delta_x) && (i < width); i++) {
            image->data[j][i].Red = buffer->data[v][u].Red;
            image->data[j][i].Green = buffer->data[v][u].Green;
            image->data[j][i].Blue = buffer->data[v][u].Blue;
            u++;
        }
        u = 0;
        v++;
    }
}

```

```

Error copypasteImageArea(BMP *image, Coordinates *copy_left, Coordinates
*copy_right, Coordinates *paste_left) {

```

```

    ImageData *buffer_data;
    size_t right_y;

    if (checkCopyCoordinates(image, copy_left, copy_right)) {
        if (checkPasteCoordinates(image, paste_left, *copy_left, *copy_right,
&right_y)) {
            buffer_data = copyImageArea(image, *copy_left, *copy_right);
            if (!buffer_data) {
                return some_kind_of_image_error;
            }
            //saveImage(buffer_image,
"C:\\Users\\Admin\\CLionProjects\\cw_bmp\\images\\generated\\Bridge_copied.bm
p");
            pasteImageArea(image, buffer_data, *paste_left, right_y);
            freeImageData(buffer_data->data, buffer_data->height);
            free(buffer_data);
            return NO_ERROR;
        }
    }

    printf("Wrong coordinates!\n");
    return NO_ERROR;
}

const char *makePath(const char *path, int part_image_num) {
    const char *new_path;
    size_t new_path_len;
    char num[9];

    new_path_len = strlen(path) + 1 + strlen(num) + 4 + 1;
    new_path = (const char *) malloc(new_path_len * sizeof(char));
    if (!new_path) {
        some_kind_of_image_error = ERROR_MEMORY;
        return NULL;
    }

    sprintf((char *) new_path, "%s/%d.bmp", path, part_image_num);

    return new_path;
}

int checkNM(BMP *image, size_t N, size_t M) {
    size_t width = image->BitmapFileInfoHeader.Width;
    size_t height = image->BitmapFileInfoHeader.Height;

    return !(N > width || M > height) && !(N == 0 || M == 0);
}

// N = X, M = Y
CoordinatesArrays *countCoordinates(BMP *image, size_t N, size_t M) {
    size_t width = image->BitmapFileInfoHeader.Width;
    size_t height = image->BitmapFileInfoHeader.Height;
    size_t X_part = width / N;
    size_t Y_part = height / M;
    size_t X_remains = width % N;
    size_t Y_remains = height % M;
    size_t prev = 0;

```

```

    CoordinatesArrays *result = (CoordinatesArrays *)
    malloc(sizeof(CoordinatesArrays));
    if (!result) {
        some_kind_of_image_error = ERROR_MEMORY;
        return NULL;
    }

    size_t *X_left = (size_t *) calloc(N, sizeof(size_t));
    size_t *Y_left = (size_t *) calloc(M, sizeof(size_t));
    size_t *X_right = (size_t *) calloc(N, sizeof(size_t));
    size_t *Y_right = (size_t *) calloc(M, sizeof(size_t));
    if (!(X_left && Y_left && X_right && Y_right)) {
        free(result);
        some_kind_of_image_error = ERROR_MEMORY;
        return NULL;
    }

    for (size_t i = 1; i < N; i++) {
        X_left[i] = prev + X_part + 1 * (X_remains != 0);
        prev = X_left[i];
        X_remains -= X_remains != 0;
    }

    prev = 0;
    for (size_t j = 1; j < M; j++) {
        Y_right[j] = prev + Y_part + 1 * (Y_remains != 0);
        prev = Y_right[j];
        Y_remains -= Y_remains != 0;
    }

    for (size_t i = 0; i < N - 1; i++) {
        X_right[i] = X_left[i + 1] - 1;
    }

    for (size_t j = 0; j < M - 1; j++) {
        Y_left[j] = Y_right[j + 1] - 1;
    }

    X_right[N - 1] = width - 1;
    Y_left[M - 1] = height - 1;

    result->x_left = X_left;
    result->y_left = Y_left;
    result->x_right = X_right;
    result->y_right = Y_right;

    return result;
}

Error splitImage(BMP *image, const char *path, size_t N, size_t M) {
    Error result_error;
    CoordinatesArrays *coordinates_arrays;
    Coordinates coordinates_pair_left;
    Coordinates coordinates_pair_right;
    ImageData *image_part_data;
    BMP *image_part;
    const char *image_part_path;
    int num = 1;

```

```

if (checkNM(image, N, M)) {
    coordinates_arrays = countCoordinates(image, N, M);
    if (!coordinates_arrays) {
        return some_kind_of_image_error;
    }
    for (size_t j = 0; j < M; j++) {
        for (size_t i = 0; i < N; i++) {
            coordinates_pair_left.x = coordinates_arrays->x_left[i];
            coordinates_pair_left.y = coordinates_arrays->y_left[j];
            coordinates_pair_right.x = coordinates_arrays->x_right[i];
            coordinates_pair_right.y = coordinates_arrays->y_right[j];

            image_part_data = copyImageArea(image, coordinates_pair_left,
coordinates_pair_right);
            if (!image_part_data) {
                freeCoordinatesArrays(coordinates_arrays);
                return some_kind_of_image_error;
            }
            image_part = generateImage(image_part_data->data,
image_part_data->width, image_part_data->height);
            if (!image_part) {
                freeCoordinatesArrays(coordinates_arrays);
                freeImageData(image_part_data->data, image_part_data-
>height);

                free(image_part_data);
                return some_kind_of_image_error;
            }
            image_part_path = makePath(path, num);
            if (!image_part_path) {
                freeCoordinatesArrays(coordinates_arrays);
                freeImageData(image_part_data->data, image_part_data-
>height);

                free(image_part_data);
                freeImage(image_part);
                return some_kind_of_image_error;
            }
            result_error = saveImage(image_part, image_part_path);
            if (result_error) {
                freeCoordinatesArrays(coordinates_arrays);
                freeImageData(image_part_data->data, image_part_data-
>height);

                free(image_part_data);
                freeImage(image_part);
                free((char *) image_part_path);
                return result_error;
            }

            freeImageData(image_part_data->data, image_part_data-
>height);

            free(image_part_data);
            freeImage(image_part);
            free((char *) image_part_path);
            num++;
        }
    }

    freeCoordinatesArrays(coordinates_arrays);
} else {
    printf("Image can't be split!\n");

```

```

    }
    return NO_ERROR;
}

void reflectImageVertically(ImageData *image_data) {
    size_t width = image_data->width;
    size_t height = image_data->height;
    RGB buffer;

    for (size_t j = 0; j < height; j++) {
        for (size_t i = 0; i < width / 2; i++) {
            buffer = image_data->data[j][i];
            image_data->data[j][i] = image_data->data[j][width - 1 - i];
            image_data->data[j][width - 1 - i] = buffer;
        }
    }
}

Error reflectImageHorizontally(ImageData *image_data) {
    size_t width = image_data->width;
    size_t height = image_data->height;
    RGB *buffer = (RGB *) malloc(width * sizeof(RGB));
    if (!buffer) {
        return ERROR_MEMORY;
    }

    for (size_t j = 0; j < height / 2; j++) {
        memcpy(buffer, image_data->data[j], width * sizeof(RGB));
        memcpy(image_data->data[j], image_data->data[height - 1 - j], width *
sizeof(RGB));
        memcpy(image_data->data[height - 1 - j], buffer, width *
sizeof(RGB));
    }

    free(buffer);
    return NO_ERROR;
}

Error reflectImageArea(BMP *image, const char *axis, Coordinates *left,
Coordinates *right) {
    int is_horizontal = strcmp(axis, "horizontal") == 0 || strcmp(axis, "h")
== 0;
    int is_vertical = strcmp(axis, "vertical") == 0 || strcmp(axis, "v") ==
0;
    ImageData *buffer_image_data;
    Error error_result;

    if (!is_horizontal && !is_vertical) {
        printf("Wrong axis!\n");
        return NO_ERROR;
    }

    if (checkCopyCoordinates(image, left, right)) {
        buffer_image_data = copyImageArea(image, *left, *right);
        if (!buffer_image_data) {
            return some_kind_of_image_error;
        }
    }
}

```

```

        if (is_horizontal) {
            error_result = reflectImageHorizontally(buffer_image_data);
            if (error_result) {
                freeImageData(buffer_image_data->data, buffer_image_data->height);
                return error_result;
            }
        } else {
            reflectImageVertically(buffer_image_data);
        }
        pasteImageArea(image, buffer_image_data, *left, right->y);
        freeImageData(buffer_image_data->data, buffer_image_data->height);
    } else {
        printf("Wrong coordinates!\n");
    }
    return NO_ERROR;
}

```

Файл main.c:

```

#include <getopt.h>
#include "errors.h"
#include "structs.h"
#include "iobmp.h"
#include "functions.h"

char *err_desc[ERROR_COUNT] = {"No fatal errors occurred\n", "It's not a BMP file\n",
                                "This BMP format isn't supported\n",
                                "File can't be opened\n",
                                "An error occurred during reading the file\n",
                                "An error occurred during writing the file\n",
                                "Not enough memory\n", "Wrong format of getopt input\n"};

Error some_kind_of_image_error = NO_ERROR;

void safeInputErrorExit(const char *input_path, const char *output_path, char *axis) {
    if (axis) {
        free(axis);
    }
    if (input_path) {
        free((char *) input_path);
    }
    if (output_path) {
        free((char *) output_path);
    }
    fputs(err_desc[ERROR_INPUT], stderr);
    exit(ERROR_INPUT);
}

void safeFuncErrorExit(Error error, const char *input_path, const char *output_path, char *axis) {
    if (axis) {
        free(axis);
    }
    free((char *) input_path);
}

```

```

    if (output_path) {
        free((char *) output_path);
    }
    fputs(err_desc[error], stderr);
    exit(error);
}

int main(int argc, char *argv[]) {
    Error result_error;
    BMP *image = NULL;

    if (argc == 1) {
        help();
        return 0;
    }

    int opt;
    int opt_index = 0;
    int func = '0';
    const char *input_path = NULL;
    const char *output_path = NULL;
    char *axis = NULL;
    Coordinates upper_left;
    int flag_upper_left = 0;
    Coordinates lower_right;
    int flag_lower_right = 0;
    Coordinates paste;
    int flag_paste = 0;
    RGB old_color;
    int flag_old_color = 0;
    RGB new_color;
    int flag_new_color = 0;
    size_t x_parts;
    int flag_x_parts = 0;
    size_t y_parts;
    int flag_y_parts = 0;

    static struct option long_options[] = {
        {"help",          0, 0, 'H'},

        {"info",          0, 0, 'I'},

        {"reflect",       0, 0, 'R'},
        {"axis",          1, 0, 'a'},
        {"upper_left",    1, 0, 'l'},
        {"lower_right",   1, 0, 'r'},

        {"coppypaste",    0, 0, 'P'},
        {"paste",         1, 0, 'p'},

        {"color",         0, 0, 'C'},
        {"old_color",     1, 0, 'c'},
        {"new_color",     1, 0, 'n'},

        {"split",         0, 0, 'S'},
        {"x_parts",       1, 0, 'x'},
        {"y_parts",       1, 0, 'y'},

        {"input",         1, 0, 'i'},

```

```

        {"output",      1, 0, 'o'},
        {0,             0, 0, 0}
    };

    while ((opt = getopt_long(argc, argv, "HIRa:l:r:Pp:Cc:n:Sx:y:i:o:",
long_options, &opt_index)) != -1) {
        printf("opt = %c\n", opt);
        switch (opt) {
            case 'H':
                help();
                return 0;
            case 'I':
                if (func != '0') {
                    safeInputErrorExit(input_path, output_path, axis);
                }
                func = 'I';
                break;
            case 'R':
                if (func != '0') {
                    safeInputErrorExit(input_path, output_path, axis);
                }
                func = 'R';
                break;
            case 'P':
                if (func != '0') {
                    safeInputErrorExit(input_path, output_path, axis);
                }
                func = 'P';
                break;
            case 'C':
                if (func != '0') {
                    safeInputErrorExit(input_path, output_path, axis);
                }
                func = 'C';
                break;
            case 'S':
                if (func != '0') {
                    safeInputErrorExit(input_path, output_path, axis);
                }
                func = 'S';
                break;
            case 'i':
                if (func != '0' && input_path == NULL) {
                    input_path = (const char *) malloc((strlen(optarg) + 1) *
sizeof(char));
                    if (!input_path) {
                        if (output_path) {
                            free((char *) output_path);
                        }
                        fputs(err_desc[ERROR_MEMORY], stderr);
                        exit(ERROR_MEMORY);
                    }
                    strcpy((char *) input_path, optarg);
                } else {
                    safeInputErrorExit(input_path, output_path, axis);
                }
                printf("input = %s\n", input_path);
                break;

```

```

        case 'o':
            if (func != 'O' && !output_path) {
                output_path = (const char *) malloc((strlen(optarg) + 1)
* sizeof(char));
                if (!output_path) {
                    if (input_path) {
                        free((char *) input_path);
                    }
                    fputs(err_desc[ERROR_MEMORY], stderr);
                    exit(ERROR_MEMORY);
                }
                strcpy((char *) output_path, optarg);
            } else {
                safeInputErrorExit(input_path, output_path, axis);
            }
            break;
        case 'a':
            if (func == 'R' && !axis) {
                axis = (char *) malloc(11 * sizeof(char));
                if (!axis) {
                    safeInputErrorExit(input_path, output_path, axis);
                }
                strcpy((char *) axis, optarg);
            } else {
                safeInputErrorExit(input_path, output_path, axis);
            }
            break;
        case 'l':
            if (!(func == 'R' || func == 'P') && !flag_upper_left &&
                sscanf(optarg, "%lu:%lu", &upper_left.x, &upper_left.y)
== 2)) {
                safeInputErrorExit(input_path, output_path, axis);
            }
            flag_upper_left++;
            break;
        case 'r':
            if (!(func == 'R' || func == 'P') && !flag_lower_right &&
                sscanf(optarg, "%lu:%lu", &lower_right.x,
&lower_right.y) == 2)) {
                safeInputErrorExit(input_path, output_path, axis);
            }
            flag_lower_right++;
            break;
        case 'p':
            if (!(func == 'P' && !flag_paste &&
                sscanf(optarg, "%lu:%lu", &paste.x, &paste.y) == 2)) {
                safeInputErrorExit(input_path, output_path, axis);
            }
            flag_paste++;
            break;
        case 'c':
            if (!(func == 'C' && !flag_old_color &&
                sscanf(optarg, "%hhu:%hhu:%hhu", &old_color.Red,
&old_color.Green, &old_color.Blue) == 3)) {
                safeInputErrorExit(input_path, output_path, axis);
            }
            flag_old_color++;
            break;
        case 'n':
            if (!(func == 'C' && !flag_new_color &&

```

```

        sscanf(optarg, "%hhu:%hhu:%hhu", &new_color.Red,
&new_color.Green, &new_color.Blue) == 3)) {
            safeInputErrorExit(input_path, output_path, axis);
        }
        flag_new_color++;
        break;
    case 'x':
        if (!(func == 'S' && !flag_x_parts &&
            sscanf(optarg, "%lu", &x_parts) == 1)) {
            safeInputErrorExit(input_path, output_path, axis);
        }
        flag_x_parts++;
        break;
    case 'y':
        if (!(func == 'S' && !flag_y_parts &&
            sscanf(optarg, "%lu", &y_parts) == 1)) {
            safeInputErrorExit(input_path, output_path, axis);
        }
        flag_y_parts++;
        break;
    default:
        safeInputErrorExit(input_path, output_path, axis);
    }
}

if (func == '0') {
    printf("No functions entered!\n");
    safeInputErrorExit(input_path, output_path, axis);
}

if (input_path) {
    image = openImage(input_path);
    if (!image) {
        safeFuncErrorExit(result_error, input_path, output_path, axis);
    }
} else {
    printf("No image path entered!\n");
    safeInputErrorExit(input_path, output_path, axis);
}

switch (func) {
    case 'I':
        result_error = info(image, input_path);
        if (result_error) {
            freeImage(image);
            safeFuncErrorExit(result_error, input_path, output_path,
axis);
        }
        break;
    case 'R':
        if (axis && flag_upper_left && flag_lower_right) {
            result_error = reflectImageArea(image, axis, &upper_left,
&lower_right);
            if (result_error) {
                freeImage(image);
                safeFuncErrorExit(result_error, input_path, output_path,
axis);
            }
        } else {

```

```

        printf("Too few arguments!\n");
        freeImage(image);
        safeInputErrorExit(input_path, output_path, axis);
    }
    break;
case 'P':
    if (flag_upper_left && flag_lower_right && flag_paste) {
        result_error = copypasteImageArea(image, &upper_left,
&lower_right, &paste);
        if (result_error) {
            freeImage(image);
            safeFuncErrorExit(result_error, input_path, output_path,
axis);
        }
    } else {
        printf("Too few arguments!\n");
        freeImage(image);
        safeInputErrorExit(input_path, output_path, axis);
    }
    break;
case 'C':
    if (flag_old_color && flag_new_color) {
        changeImageColor(image, old_color, new_color);
    } else {
        printf("Too few arguments!\n");
        freeImage(image);
        safeInputErrorExit(input_path, output_path, axis);
    }
    break;
case 'S':
    if (output_path && flag_x_parts && flag_y_parts) {
        result_error = splitImage(image, output_path, x_parts,
y_parts);
        if (result_error) {
            freeImage(image);
            safeFuncErrorExit(result_error, input_path, output_path,
axis);
        }
    } else {
        printf("Too few arguments!\n");
        freeImage(image);
        safeInputErrorExit(input_path, output_path, axis);
    }
    break;
default:
    break;
}

if (func != 'S') {
    if (output_path) {
        result_error = saveImage(image, output_path);
    } else {
        result_error = saveImage(image, input_path);
    }
    if (result_error) {
        freeImage(image);
        safeFuncErrorExit(result_error, input_path, output_path, axis);
    }
}
}

```

```
    freeImage(image);
    free((char *) input_path);
    if (output_path) {
        free((char *) output_path);
    }

    return 0;
}
```

Файл Makefile:

```
all: main.o iobmp.o functions.o
    gcc main.o iobmp.o functions.o -o cw_bmp

main.o: main.c errors.h structs.h iobmp.h functions.h
    gcc -c main.c

iobmp.o: iobmp.h errors.h structs.h
    gcc -c iobmp.c

functions.o: functions.h errors.h structs.h iobmp.h
    gcc -c functions.c
```